

Linux I/O Streams and Text Processing Commands - Complete Notes

1. stdout (Standard Output)

Concept: Standard output (stdout) is the default output stream where commands send their results, typically to the screen.

Basic Output Redirection

Redirect to file (overwrite):

```
echo Hello World > peanuts.txt
```

- Creates `peanuts.txt` with "Hello World"
- `>` overwrites existing file content

Append to file:

```
echo Hello World >> peanuts.txt
```

- Adds "Hello World" to the end of `peanuts.txt`
- Creates file if it doesn't exist

2. stdin (Standard Input)

Concept: Standard input (stdin) is where commands receive their input, typically from keyboard or files.

Input Redirection

Redirect file as input:

```
cat < peanuts.txt > banana.txt
```

- Uses `peanuts.txt` as input to `cat`
- Outputs result to `banana.txt`

- `<` redirects stdin from file

3. stderr (Standard Error)

Concept: Standard error (stderr) is a separate stream for error messages, distinct from stdout.

File Descriptors

- `stdin = 0`
- `stdout = 1`
- `stderr = 2`

Error Redirection Examples

Redirect stderr only:

```
ls /fake/directory 2> peanuts.txt
```

- Only error messages go to file
- Normal output still goes to screen

Redirect both stdout and stderr:

```
ls /fake/directory > peanuts.txt 2>&1
```

- Redirects stdout to file, then stderr to wherever stdout goes

Shorter syntax for both:

```
ls /fake/directory &> peanuts.txt
```

Discard stderr:

```
ls /fake/directory 2> /dev/null
```

- Sends errors to `/dev/null` (discards them)

4. pipe and tee

Pipe Operator `|`

Concept: Connects stdout of one command to stdin of another.

```
ls -la /etc | less
```

- Output of `ls` becomes input for `less`

tee Command

Concept: Writes output to both stdout and a file simultaneously.

```
ls | tee peanuts.txt
```

- Shows `ls` output on screen AND saves to `peanuts.txt`

5. env (Environment Variables)

Viewing Environment Variables

Display specific variable:

```
$ echo $HOME
# Output: /home/pete

$ echo $USER
# Output: pete
```

Display all variables:

```
env
```

PATH variable example:

```
$ echo $PATH
# Output: /usr/local/sbin:/usr/local/bin:/usr/sbin:/bin
```

6. cut - Extract Text Portions

Extract by Character Position

```
$ echo 'The quick brown; fox jumps over the lazy dog' > sample.txt
$ cut -c 5 sample.txt
# Output: q (5th character)
```

Extract by Field (TAB-delimited)

```
$ cut -f 2 sample.txt
# Output: dog (assuming TAB between "lazy" and "dog")
```

Extract with Custom Delimiter

```
$ cut -f 1 -d ";" sample.txt
# Output: The quick brown
```

7. paste - Merge Lines

Sample File Creation

```
sample2.txt:
The
quick
brown
fox
```

Merge Lines with Default TAB Delimiter

```
$ paste -s sample2.txt
# Output: The    quick    brown    fox
```

Merge with Custom Delimiter

```
$ paste -d ' ' -s sample2.txt
# Output: The quick brown fox
```

8. head - View First Lines

Default (First 10 Lines)

```
head /var/log/syslog
```

Custom Line Count

```
head -n 15 /var/log/syslog
```

9. tail - View Last Lines

Default (Last 10 Lines)

```
tail /var/log/syslog
```

Line Count

```
tail -n 10 /var/log/syslog
```

Follow File Changes

```
$ tail -f /var/log/syslog  
# Continuously shows new lines as they're added
```

10. expand and unexpand - Tab/Space Conversion

Convert TABs to Spaces

```
expand sample.txt  
expand sample.txt > result.txt
```

Convert Spaces to TABs

```
unexpand -a result.txt
```

11. join and split - File Operations

Join Files by Common Field

Sample Files:

file1.txt:	file2.txt:
1 John	1 Doe
2 Jane	2 Doe
3 Mary	3 Sue

Basic Join:

```
$ join file1.txt file2.txt
# Output:
# 1 John Doe
# 2 Jane Doe
# 3 Mary Sue
```

Join Different Fields:

file1.txt:	file2.txt:
John 1	1 Doe
Jane 2	2 Doe
Mary 3	3 Sue

```
$ join -1 2 -2 1 file1.txt file2.txt
# Output:
# 1 John Doe
# 2 Jane Doe
# 3 Mary Sue
```

Split Files

```
$ split somefile
# Creates multiple files (x**, xab, xac, etc.) with 1000 lines each
```

12. sort - Sort Lines

Sample File:

```
file1.txt:
dog
cow
cat
```

```
elephant  
bird
```

Basic Sort

```
$ sort file1.txt  
# Output:  
# bird  
# cat  
# cow  
# dog  
# elephant
```

Reverse Sort

```
$ sort -r file1.txt  
# Output:  
# elephant  
# dog  
# cow  
# cat  
# bird
```

Numerical Sort

```
sort -n file1.txt
```

13. tr (Translate) - Character Translation

Convert Case

```
$ tr a-z A-Z  
hello  
# Output: HELLO
```

14. uniq (Unique) - Handle Duplicates

Sample File:

```
reading.txt:
book
book
paper
paper
article
article
magazine
```

Remove Adjacent Duplicates

```
$ uniq reading.txt
# Output:
# book
# paper
# article
# magazine
```

Count Occurrences

```
$ uniq -c reading.txt
# Output:
# 2 book
# 2 paper
# 2 article
# 1 magazine
```

Show Only Unique Lines

```
$ uniq -u reading.txt
# Output: magazine
```

Show Only Duplicate Lines

```
$ uniq -d reading.txt
# Output:
# book
# paper
# article
```

Handle Non-Adjacent Duplicates

```
$ sort reading.txt | uniq
# Sorts first, then removes duplicates
```

15. wc and nl - Counting

Word Count

```
$ wc /etc/passwd
# Output: 96      265      5925 /etc/passwd
# (lines) (words) (bytes) (filename)
```

Count Specific Fields

```
$ wc -l /etc/passwd # Lines only
# Output: 96

$ wc -w /etc/passwd # Words only
$ wc -c /etc/passwd # Bytes only
```

Number Lines

Sample File:

```
file1.txt:
i
like
turtles
```

```
$ nl file1.txt
# Output:
# 1. i
# 2. like
# 3. turtles
```

16. grep - Search Text Patterns

Basic Search

```
$ grep fox sample.txt
# Finds and displays lines containing "fox"
```

Case-Insensitive Search

```
grep -i somepattern somefile
```

Combine with Pipes

```
$ env | grep -i User
# Search environment variables for "User"
```

Regular Expressions

```
$ ls /somedir | grep '.txt$'
# Find all files ending with .txt
```

Key Concepts Summary

- **I/O Streams:** stdin (0), stdout (1), stderr (2)
- **Redirection:** `>` (overwrite), `>>` (append), `<` (input), `2>` (stderr)
- **Pipes:** `|` connects commands together
- **File Descriptors:** Numerical references to streams
- **Text Processing:** Essential for data manipulation and analysis
- **Command Chaining:** Combine multiple commands for complex operations